RESEARCH OWASP WSTG
Chapter 1: Background introduction:
I, Context Background

II, Introduction to the OWASP WSTG

Chapter 2: Theoretical Research
I, OWASP WSTG framework

Based on the document, the WSTG have 2 main part:
+ The web security testing framework
+ Web application security testing
1, The web security framework
This section of the document describes the activities that take place in a SDLC:
+ Before developments begins
+ During definition and design
+ During development
+ During Deployment
+ During maintenance and operations
2, Web application security testing
The web application security testing is approached in a way that combines both Risk-based and Human intelligence led, which is using both automation tools and human intelligence . The OWASP WSTG method is based on black box testing.The testing guide has all the possible testing techniques for all possible vulnerabilities. Testing are categorized into:
+ Passive testing: Understand the application's logic and explores the application as a user
+ Active testing: During active testing, a tester begins to use the methodologies that are provided by the documents. The set of activities have been split into 12 categories :
+ Information Gathering
+ Configuration and Deployment Management Testing
+ Identity Management Testing
+ Authentication Testing
+ Authorization Testing
+ Session Management Testing
+ Input Validation Testing
+ Error Handling
+ Cryptography
+ Business Logic Testing
+ Client-side Testing
+ API Testing

II, The Philosophy of Standardized Testing

In the fast-paced world of software development, it is crucial to ensure the quality and functionality of applications. Manual testing, while thorough, is time-consuming and prone to

human error, especially for repetitive tasks. This is where testing frameworks come in. Web testing framework standards exist to transition from chaotic, manual, and unmaintainable testing scripts to a structured, efficient, and automated process.

Web testing framework provide the tester with these features:
+ Maintainability: Requirement changes most often. To maintain the test cases with the changing requirements, the framework gives testers the additional benefits to make the changes more efficiently. Also, anyone can add more test cases or modify existing test cases if needed.
+ Reusability: Frameworks give the power tester to make their test automation process in a more non-redundant way and reuse the automated test cases.
+ Reliability: A standard provides a common language and structure for testing, which helps when multiple team members work on the same project. Moreover, the testing framework is designed to address all the possible problems in order to minimize human error.
+ Consistency: The testing guide ensures it is well documented and instructs the tester detail step by step so that 2 testers could get similar results on the same test targets.

As mentioned earlier, The OWASP WSTG applies both automation tools and human intelligence. Automation testing tools software used to execute pre-scripted tests on software. Because automation tools are pre-scripted / hard coded so it only does the same job on every run time and makes the test become very static and inflexible. On the other hand, manual testing based on human intelligence leads is very dynamic and flexible and it can adjust based on the context of the given target web application. But humans are not machines, they can make mistakes and can miss out on some critical flaws during the testing process. Moreover , manual testing is time consuming and very slow and inefficient. By combining both of them and guiding how to combine both Manual testing and automation tools, the testing guide maximizes the efficiency and reliability of the testing process. Moreover, the guide also provide the tester a remediation ways for each specific vulnerabilities so that the guide could fit well in the SDLC.

III, The Risk-Based Approach

The OWASP WSTG focuses mainly on testing all possible known vulnerabilities and threats that were discovered in the past to see if any vulnerabilities exist on the system or not. Especially, the guide focuses on the flaw that could cause real damage and have high business impact rather than look for specific bugs in the system.

By prioritizing high-impact vulnerabilities, the WSTG ensures that organizations focus resources on security issues that matter most rather than minor risks. The guide aims to simulate real-world attacks to identify vulnerabilities that could lead to threats like: Data theft and leakage, Unauthorized access control, Service disruption, Business Logic failures,... These threats could cause massive damage.

On the other hand, the WSTG strongly focuses on finding real technical vulnerabilities that connect or have related to an actual business risk rather than finding minor bugs that have no

connections to a business risk. A key component of the guide is to safely verify the real impact of findings, showing the concrete danger to data.It assists in prioritizing remediation by estimating the severity of risks, preventing teams from being distracted by minor issues. Moreover, The WSTG covers areas that automated scanners often miss, focusing deeply on manual testing to find subtle vulnerabilities that lead to significant breaches

IV, Defense in Depth

With the development and expansion of web applications nowadays, web application is no longer a lone software run in a single computer that responds every time it receives a HTTP request. Web application has evolved into a complex structured system that consists of many components of different software that are linked together run in a cluster of hundred computers to serve millions of users and no longer be a single process in a computer. These expand the attack vector and increase unknown vulnerabilities that lay inside different components of the web server that can be abused and causing damage at any time.
Web applications are now becoming a “stack” , a combination of using many different software like databases, web engines, firewalls, … This leads to chain effects, where every component is dependent on each other, a break in a single chain could cause a disruption for the whole system. In security , one chain link is exposed to attack and could lead other chain links to become vulnerable, a vulnerability at the infrastructure layer (e.g., misconfigured firewall) can expose the application layer, even if the application code is secure. Because of this, The WSTG provided a holistic, multi-layered approach that is required to identify vulnerabilities across the infrastructure, application platform, and code, rather than focusing solely on the application code. As we mentioned above, the WSTG includes 12 testing categories that span all of these layers and possible attack surfaces.

Moreover, the WSTG also covered all the attack surface and preventing “silent” vulnerabilities that may appear in the web application

V, The Software Development Life Cycle integration

The integration of the OWASP Web Security Testing Guide (WSTG) into the Software Development Life Cycle (SDLC) represents a shift from reactive security to proactive "Security by Design." Historically, security testing was treated as an isolated, final hurdle performed just before an application’s release. This "perimeter-defense" mindset often resulted in the discovery of fundamental architectural flaws at a stage where remediation was both prohibitively expensive and technically disruptive. The WSTG provides the necessary framework to move beyond this outdated model by advocating for security checkpoints throughout every phase of the development journey.

A core tenet of modern application security supported by the WSTG is the concept of "Shifting Left." This philosophy posits that the earlier a vulnerability is identified in the SDLC, the lower the cost and effort required to fix it. By utilizing the WSTG’s methodologies during the requirements and design phases, architects can anticipate potential threats—such as broken

access control or insecure data storage—before a single line of code is written. During the development phase, the guide serves as a manual for secure coding practices, allowing developers to perform "unit-level" security tests that align with the guide's testing criteria for input validation and session management.

Ultimately, integrating the WSTG into the SDLC ensures that security is a shared responsibility rather than a siloed task. It provides a common language for developers, QA engineers, and security professionals to collaborate. By embedding the guide's systematic checks into the development lifecycle, organizations move away from "security as a bottleneck" and toward a culture of continuous security, where the robustness of the application is verified and strengthened at every iteration of its growth.

Chapter 3: Methodology Breakdown
I, Framework Structure Overview
II, Deep Dive into Key Testing Categories
1, Information Gathering
2, Authentication testing
3, Authorizing testing
4, Input validation testing
5, Business Logic testing
III, The Reporting and Scoring Mechanism